

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5117

NAME: G. BALASUBRAMANIAN

REG.NO:192521001

DEPARTMENT: B.TECH. IT

ASSIGNMENT: C02 – AT1

FACULTY NAME: DR. R. DHARANI

1. RSA Encryption Analysis

Given:

- $(p = 11)$
 - $(q = 13)$
 - $(e = 7)$
 - Message $(M = 9)$
-

a. Compute n and $\phi(n)$

Step 1: Calculate n

$$\begin{aligned}n &= p \times q \\n &= 11 \times 13 = 143\end{aligned}$$

Step 2: Calculate Euler's Totient Function

$$\begin{aligned}\phi(n) &= (p - 1)(q - 1) \\ \phi(n) &= (11 - 1)(13 - 1) \\ \phi(n) &= 10 \times 12 = 120\end{aligned}$$

Answer:

- $n = 143$
- $\phi(n) = 120$

b. Choose $e = 7$ and find the private key d

The private key d satisfies:

$$d \times e \equiv 1 \pmod{\phi(n)}$$

Substitute the values:

$$d \times 7 \equiv 1 \pmod{120}$$

Find the multiplicative inverse of 7 modulo 120.

$$\begin{aligned}7 \times 103 &= 721 \\ 721 \div 120 &= 6 \text{ remainder } 1\end{aligned}$$

Hence,

$$721 \equiv 1 \pmod{120}$$

Therefore,

$$\boxed{d = 103}$$

Keys

- **Public Key:** (7, 143)
- **Private Key:** (103, 143)

b) Encrypt the message $M = 9$

- RSA Encryption Formula:
- $C = M^e \pmod{n}$

$$C = 9^7 \pmod{143}$$

- Calculate:
- $9^2 = 81$

$$\begin{aligned} 9^4 &= 81^2 = 6561 \equiv 126 \pmod{143} \\ 9^7 &= 9^4 \times 9^2 \times 9 \\ &= 126 \times 81 \times 9 \\ 126 \times 81 &= 10206 \\ 10206 \pmod{143} &= 53 \\ 53 \times 9 &= 477 \\ 477 \pmod{143} &= 48 \end{aligned}$$

- Therefore,
- $\boxed{C = 48}$

Decrypt the ciphertext and verify the result

RSA Decryption Formula:

$$\begin{aligned} M &= C^d \pmod{n} \\ M &= 48^{103} \pmod{143} \end{aligned}$$

Using modular exponentiation,

$$48^{103} \pmod{143} = 9$$

Thus,

$$M = 9$$

The decrypted message is the same as the original message.

Verification:

Original Message = 9

Decrypted Message = 9

e. Analyze how changing e affects security

The value of e is the **public exponent** in RSA.

Effects of Changing e

- **Smaller e** (e.g., 3 or 5)
 - Encryption is faster.
 - If used without proper padding, it may be vulnerable to certain attacks.
 - Less secure in some situations.
- **Larger e** (e.g., 65537)
 - Provides better security against known attacks.
 - Encryption is slightly slower.
 - It is the most commonly used value because it offers a good balance between security and performance.
- **Requirement**
 - The value of e must satisfy:

$$1 < e < \phi(n)$$

and

$$\gcd(e, \phi(n)) = 1$$

so that a valid private key d exists.

2. Block Cipher Mode Evaluation

Given:

- Plaintext Blocks:
 - $P_1 = 1010$
 - $P_2 = 1010$
 - $P_3 = 1111$
 - $P_4 = 1010$
- Key (K) = 1100
- IV = 0000 (for CBC)

Assumption

Use a simple block cipher where:

$$\text{Ciphertext} = \text{Plaintext} \oplus \text{Key}$$

(where $\oplus$ denotes XOR operation)

a. Encrypt using ECB and CBC modes

1. ECB (Electronic Codebook) Mode

Formula:

$$C_i = P_i \oplus K$$

Block 1

$$1010 \oplus 1100 = 0110$$

Block 2

$$1010 \oplus 1100 = 0110$$

Block 3

$$1111 \oplus 1100 = 0011$$

Block 4

$$1010 \oplus 1100 = 0110$$

Block Plaintext ECB Ciphertext CBC Ciphertext

P ₁	1010	0110	0110
P ₂	1010	0110	0000
P ₃	1111	0011	0011
P ₄	1010	0110	0101

Observation

- In **ECB**, identical plaintext blocks (**1010**) produce the **same ciphertext (0110)**.
- In **CBC**, identical plaintext blocks produce **different ciphertext values** because each block depends on the previous ciphertext block.

Analyze which mode hides patterns better and explain why

ECB Mode

- Each plaintext block is encrypted independently.
- Repeated plaintext blocks generate identical ciphertext blocks.
- Patterns in the original data remain visible.
- Hence, ECB is **less secure** for messages containing repeated data.

CBC Mode

- Each plaintext block is first XORed with the previous ciphertext block before encryption.
- The first block uses the Initialization Vector (IV).
- Even if plaintext blocks are identical, their ciphertext blocks are different because the previous ciphertext changes for every block.
- This hides patterns and provides better confidentiality.

3.

DES vs 3-DES vs Blowfish Performance Study

Given:

- **DES**
 - Key Size = **56 bits**
 - Encryption Time = **2 ms/block**
- **3-DES**
 - Key Size = **168 bits**
 - Encryption Time = **6 ms/block**
- **Blowfish**
 - Key Size = **128 bits**
 - Encryption Time = **1 ms/block**

- **Total Data = 1000 Blocks**

•

- **Calculate the total encryption time for each algorithm**

- **Formula**

- Total Encryption Time = Time per Block $\times$ Number of Blocks

- **DES**

- $2 \text{ ms} \times 1000 = 2000 \text{ ms}$

2000 ms = 2 seconds

- **3-DES**

- $6 \text{ ms} \times 1000 = 6000 \text{ ms}$

6000 ms = 6 seconds

- **Blowfish**

- $1 \text{ ms} \times 1000 = 1000 \text{ ms}$

1000 ms = 1 second

Analyze the trade-offs between security and performance

DES

- Uses a **56-bit key**, which is too small by modern standards.
- Fast encryption compared to 3-DES.
- Vulnerable to brute-force attacks.
- **Performance:** Good
- **Security:** Low

3-DES

- Uses an effective **168-bit key** by applying DES three times.
- Much stronger security than DES.
- Slowest among the three algorithms because encryption is performed three times.
- **Performance:** Low
- **Security:** High

Blowfish

- Uses a **128-bit key** (or larger in practice).
- Provides strong security with efficient encryption.
- Fastest among the three algorithms.
- Suitable for applications requiring both speed and security.
- **Performance:** High
- **Security:** High

Recommend the best algorithm for secure real-time communication

Recommended Algorithm: Blowfish

Justification

- Provides **strong security** with a **128-bit key**.
- Requires only **1 second** to encrypt 1000 blocks, making it the fastest among the three.
- Consumes less processing time, making it suitable for **real-time communication** such as secure messaging, online transactions, and network communication.
- Offers a better balance between **security** and **performance** than DES and 3-DES.

Conclusion

For encrypting **1000 data blocks**:

- **DES** takes **2 seconds** but offers weak security.
- **3-DES** takes **6 seconds** and provides strong security but is too slow for real-time applications.
- **Blowfish** takes **1 second**, provides high security, and offers the best overall performance.

Therefore, Blowfish is the most suitable algorithm for secure real-time communication because it combines strong security with the fastest encryption speed.

4. Diffie-Hellman Key Exchange Analysis

Given

- Prime number: $p = 23$
- Generator: $g = 5$
- User A's private key: $a = 6$
- User B's private key: $b = 15$
- Compute the Public Keys for A and B
- **Formula**
- Public Key = $g^{\text{Private Key}} p$
- **User A's Public Key**
- $A = 5^6 23$

$$5^2 = 25 \equiv 2 \pmod{23}$$

$$5^4 = 2^2 = 4$$

$$5^6 = 5^4 \times 5^2 = 4 \times 2 = 8$$

$$\boxed{A = 8}$$

- **User B's Public Key**
- $B = 5^{15} 23$
- Using modular arithmetic,
- $5^{15} 23 = 19$

$$\boxed{B = 19}$$

User A computes

- $K = B^a p$

$$K = 19^6 23$$

$$19^2 = 361 \equiv 16 \pmod{23}$$

$$19^4 = 16^2 = 256 \equiv 3 \pmod{23}$$

$$19^6 = 19^4 \times 19^2 = 3 \times 16 = 48$$

$$48 23 = 2$$

$$\boxed{K = 2}$$

$$K = 8^{15} 23$$

Using modular arithmetic,

$$8^{15} 23 = 2$$

$$\boxed{K = 2}$$

c. Demonstrate how a Man-in-the-Middle (MITM) attacker could alter the exchange

Suppose an attacker Eve is between User A and User B.

Step 1

User A sends public key 8 to User B.

Step 2

Eve intercepts the message and does not forward 8.

Instead, Eve sends her own public key 13 to User B.

Step 3

User B sends public key 19 to User A.

Step 4

Eve intercepts this message and replaces it with 17 before sending it to User A.

Result

- User A believes 17 belongs to User B.
- User B believes 13 belongs to User A.
- User A establishes a secret key with Eve.
- User B establishes another secret key with Eve.
- Eve can decrypt messages, read or modify them, and then re-encrypt them before forwarding them.

Thus, both users think they are communicating securely, but all communication passes through the attacker.

Suggest a secure method to prevent the attack

The Man-in-the-Middle attack can be prevented by authenticating the public keys.

Methods to Prevent MITM

1. Digital Certificates

- **Public keys are verified using certificates issued by a trusted Certificate Authority (CA).**
- **Ensures the public key belongs to the intended user.**

2. Digital Signatures

- **Each user signs their public key with their private key.**
- **The receiver verifies the signature before accepting the key.**

3. TLS/SSL Protocol

- **Uses authenticated Diffie-Hellman along with digital certificates.**
- **Prevents attackers from replacing public keys during the exchange.**

Using $p = 23$ and $g = 5$:

- **User A's Public Key = 8**
- **User B's Public Key = 19**
- **Shared Secret Key = 2**

Although Diffie-Hellman securely establishes a shared key over a public network, it is vulnerable to Man-in-the-Middle attacks if public keys are not authenticated.

Using digital certificates, digital signatures, or TLS/SSL ensures that the exchanged public keys are genuine, preventing attackers from intercepting or altering the communication.

5.

ECC vs RSA Comparison

Given

- **RSA key size = 1024 bits**
- **ECC key size = 160 bits** (equivalent security to RSA 1024-bit)
- **Device:** IoT device with limited processing capacity

Analyze the computational cost for both systems

RSA

- RSA uses a **1024-bit key**, requiring large integer arithmetic.
- Encryption, decryption, and key generation involve complex computations.
- Execution time is higher because of the larger key size.
- Therefore, RSA has a **high computational cost** and is slower on resource-constrained devices.

ECC

- ECC uses a **160-bit key** to provide the same security level as RSA 1024-bit.
- It performs fewer mathematical operations due to the smaller key size.
- Encryption, decryption, and key generation are faster.
- Therefore, ECC has a **lower computational cost** and is more efficient for IoT devices.

Estimate which consumes less power and memory

RSA

- Larger keys require more memory for key storage.
- More CPU operations increase energy consumption.
- Results in **higher power consumption** and **higher memory usage**.

ECC

- Smaller keys require less storage space.
- Fewer computations reduce CPU usage.
- Results in **lower power consumption** and **lower memory usage**.

Estimate which consumes less power and memory

RSA

- Larger keys require more memory for key storage.
- More CPU operations increase energy consumption.
- Results in **higher power consumption** and **higher memory usage**.

ECC

- Smaller keys require less storage space.
- Fewer computations reduce CPU usage.
- Results in **lower power consumption** and **lower memory usage**.

• Conclusion

- For an IoT device with limited processing capacity, **Elliptic Curve Cryptography (ECC)** is the best choice. It delivers the **same security as RSA 1024-bit** with a **160-bit key**, while reducing computational cost, memory usage, and power consumption. Therefore, **ECC is more efficient and better suited for secure communication in resource-constrained IoT devices**.